

Pulsewatch vs. Uptime Kuma

Architectural Pattern, Data Scaling & Production Gap Analysis Report

Stack: FastAPI | Python | SQLAlchemy | Supabase PostgreSQL | React / Vite | APScheduler

1. Executive Summary

This report provides an in-depth technical benchmark comparing **Pulsewatch** against the industry-standard open-source monitoring tool **Uptime Kuma** (louislam/uptime-kuma). While Uptime Kuma is optimized as a lightweight, single-node blackbox monitor using local SQLite storage, Pulsewatch is built with a **multi-tenant cloud-native foundation** (FastAPI + Supabase PostgreSQL) and a **hybrid telemetry architecture** combining external availability polling with an in-host metric collection agent (metric_agent.py). This gives Pulsewatch significant advantages in multi-tenant SaaS capabilities and long-term trend analysis, while revealing key operational gaps in non-blocking async execution, alerting dispatch, and flap resilience.

2. Core Architecture & Feature Comparison Matrix

Dimension	Pulsewatch (Current State)	Uptime Kuma (Industry Standard)
Backend Engine	Python (FastAPI + SQLAlchemy + APScheduler)	Node.js (Express + Socket.IO + Redbean / Knex)
Database Architecture	Managed Cloud PostgreSQL (Supabase) with strict relational models	Localized SQLite (Default, WAL mode) or single-node MariaDB
Multi-Tenancy & Auth	Full Multi-Tenant isolation (JWT Bearer tokens + Email 2FA OTP)	Single-Tenant instance by default (Multi-user RBAC in v2.0)
Monitoring Telemetry	Hybrid: HTTP Reachability + Host OS CPU & RAM Agent (Port 8001)	Blackbox Only: Latency, HTTP codes, Ping, DNS, TCP, Docker
Data Retention Strategy	Two-Tier Lifecycle: Raw high-res (30d) & Hourly Rollup (15mo)	Single Heartbeat table with sliding window time-based pruning
Polling Execution	Synchronous sequential loop across all targets (Blocking risk)	Asynchronous, isolated per-monitor event loop timers
Notification Engine	Email OTP only (No down/recovery alert dispatchers)	90+ Alert channels (Discord, Slack, Telegram, Webhooks, SMS)
Flap & Retry Handling	Instant status toggle on 1 failure (No debounce or retry count)	Configurable retries (e.g. 3 consecutive failures before alert)
Status Pages	Private authenticated dashboard only	Built-in public/private custom domain status pages with incidents
Realtime UI Updates	Client-side REST polling (setInterval / manual fetch)	Bi-directional WebSockets (Socket.IO) push on every heartbeat

3. Data Architecture & Scalability Breakdown

Pulsewatch (Cloud PostgreSQL + Two-Tier Lifecycle):

- **Multi-Tenant Relational Schema:** Pulsewatch separates tenants cleanly at the database layer using indexed user_id foreign keys across Server and otp_codes tables.
- **Two-Tier Storage Pipeline:** Raw 30-second heartbeats are retained in metrics_raw for 30 days. A nightly APScheduler cron job (process_historical_data) aggregates these records into metrics_hourly (averaging CPU & RAM), allowing 15 months of historical charting with minimal database I/O.
- **Cloud Concurrency & Indexing:** Utilizing Supabase PostgreSQL enables connection pooling (PgBouncer), table partitioning, and seamless horizontal scaling without database lock contention.

Uptime Kuma (Local SQLite Model):

- **Single-Node Simplicity:** Stores monitor definitions and heartbeats inside a local kuma.db file using SQLite WAL mode.

- **Scaling Limitations:** High monitor counts (e.g. 500+ monitors checking every 20s) encounter SQLite write-lock contention, requiring heavy VACUUM maintenance and posing data loss risks if container storage is improperly mapped.

4. Core Monitoring Logic & Safety Analysis

Pulsewatch's Hybrid Model:

Pulsewatch deploys a lightweight host daemon (`metric_agent.py` via `install.sh`) on port 8001 to report system CPU and RAM usage via `psutil` alongside endpoint health. However, the polling engine currently has critical safety vulnerabilities:

CRITICAL SAFETY VULNERABILITIES IN CURRENT POLLING LOGIC:

- Synchronous Blocking Execution:** `monitoring_job()` runs a single sequential loop over all servers using `requests.get()` with 5s + 3s timeouts. If 20 servers become unreachable, the loop blocks for 160 seconds, freezing `APScheduler` and missing future 30-second cycles.
- Alert Flapping (No Retry/Debounce):** A single dropped packet immediately sets `is_active = 0`, creating false positive downtime.
- Unsecured Metric Agent:** `metric_agent.py` binds to 0.0.0.0:8001 with `CORS *` without authentication headers or TLS encryption.

5. Gap Analysis: Missing Production Features

To reach full production maturity and compete with tools like Uptime Kuma, Pulsewatch must implement the following six foundational capabilities:

- **1. Multi-Channel Notification Engine:** Pulsewatch currently only uses email for login OTPs. It lacks automated alert triggers on state transitions (UP → DOWN, DOWN → UP) for Discord, Slack, Telegram, Webhooks, PagerDuty, or Email.
- **2. Asynchronous Non-Blocking Poller:** Replace synchronous requests with `httpx.AsyncClient` and `asyncio.gather()` to execute concurrent probes across hundreds of servers concurrently in sub-second time.
- **3. Flap Prevention & Retry Thresholds:** Add `retry_count` and `max_retries` (e.g., 3 consecutive failed checks before declaring an incident) to prevent false alerts caused by transient network blips.
- **4. Public & Branded Status Pages:** Allow users to generate public read-only status dashboards with custom CNAME domains and incident announcement timelines.
- **5. Agent Authentication & TLS Security:** Secure `metric_agent.py` with pre-shared bearer tokens (Authorization: Bearer <TOKEN>) and optional HTTPS support.
- **6. SSL Certificate Expiration Monitoring:** Inspect TLS handshakes on monitored HTTPS targets and alert administrators 30, 14, and 7 days prior to SSL certificate expiration.

6. Recommended Implementation Roadmap

Phase	Target Milestone	Core Deliverables & Actions
Phase 1 (Immediate)	Engine Safety & Async Polling	Migrate <code>monitoring_job()</code> to <code>httpx.AsyncClient</code> with <code>asyncio.gather()</code> ; add retry debounce logic (3 consecutive failures).
Phase 2 (Short-Term)	Alerting & Webhook Dispatcher	Create <code>NotificationChannel</code> table (Discord, Slack, Webhook, Email); trigger automated dispatches on state transitions.
Phase 3 (Mid-Term)	Agent Hardening & SSL Checks	Add shared token auth to <code>metric_agent.py</code> ; add TLS certificate validity and expiration checking to target probes.
Phase 4 (Long-Term)	Public Status Pages & WebSockets	Implement public status page sub-routes and replace REST polling in React dashboard with FastAPI WebSockets / Server-Sent Events (SSE).